System Security report

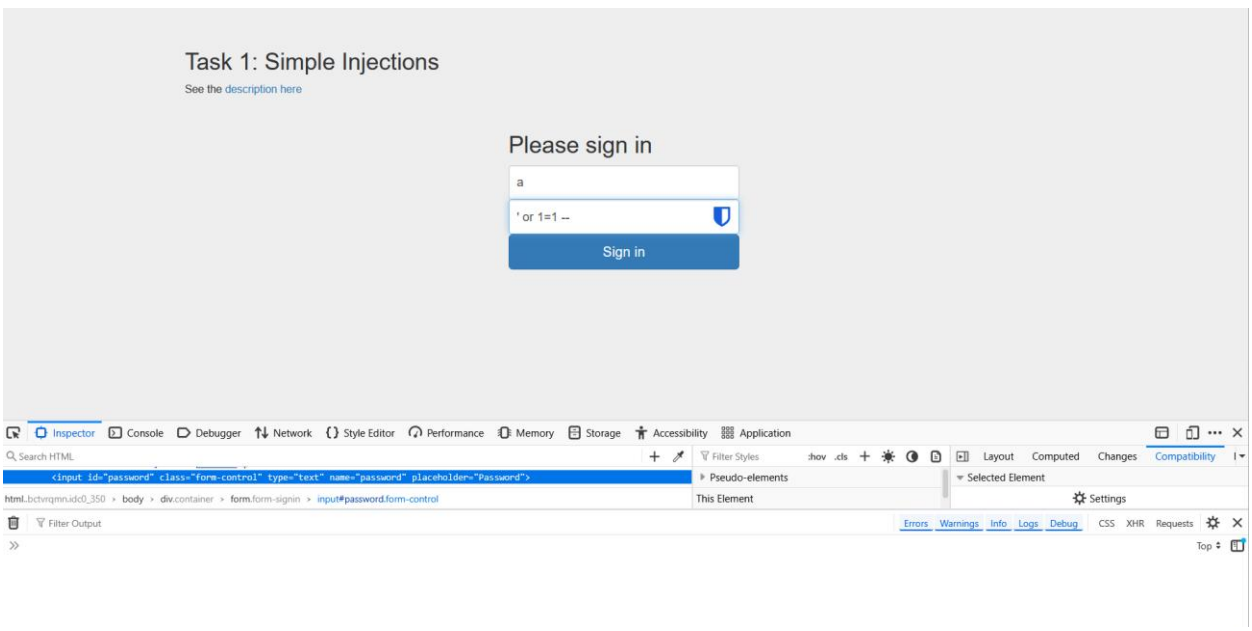
Task 1: simple injection

Problem: bypass the authentication mechanism of a login page. The application was vulnerable as it directly concatenated user input for mail and password into a SQL query, with no checks, sanitizations nor defenses, allowing for a classic SQLi attack

Method used: A tautology-based injection. I used a universally true statement (OR 1=1) and commented out the remainder of the original query. The payload is `' or 1=1 --` . *(note that in general there is a space at the end of the query but it doesn't get highlighted)*

The resulting SQL query executed by the server became

```
SELECT mail, password FROM users WHERE mail = 'task1' AND password = '' or 1=1 -- .
```

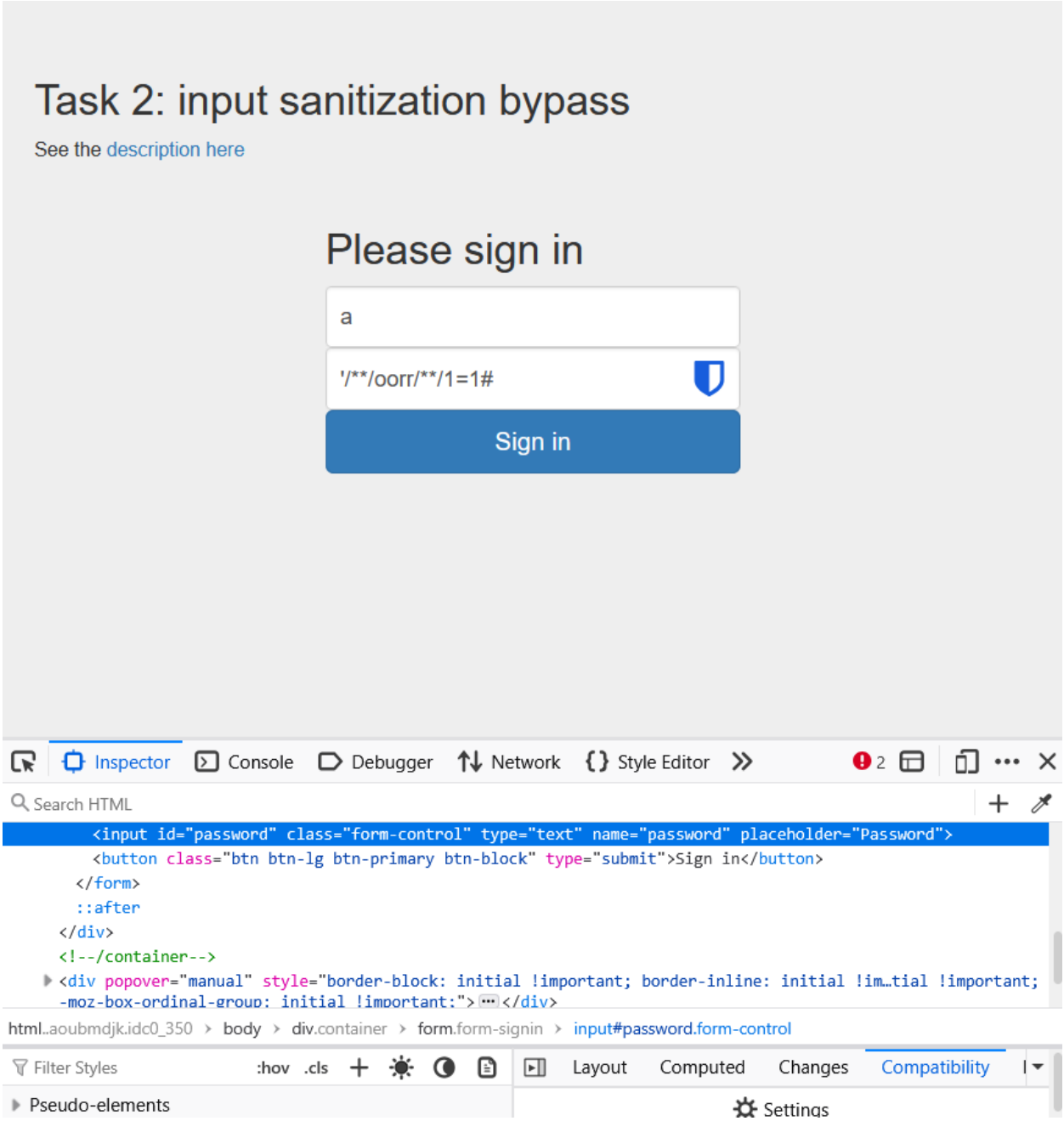


Task 2: Input sanitization bypass

Problem: bypass authentication on a site protected by a weak sanitization. This filter removed specific keywords and characters, without recursion.

Method Used: I exploited the fact that it wasn't recursive to bypass the control over the 'or'; empty comments (/**/) were used as whitespace to bypass space filtering, and the # character was used to comment out the rest of the query, bypassing the control over the '-'. The payload is: `'/**/oorr/**/1=1#` . Filter removes *or* from *oorr*, leaving *or* and treats /**/ as whitespace, resulting in `' or 1=1#` . So, again, I used a tautology query attack

(the resulting query was in fact the same as above).

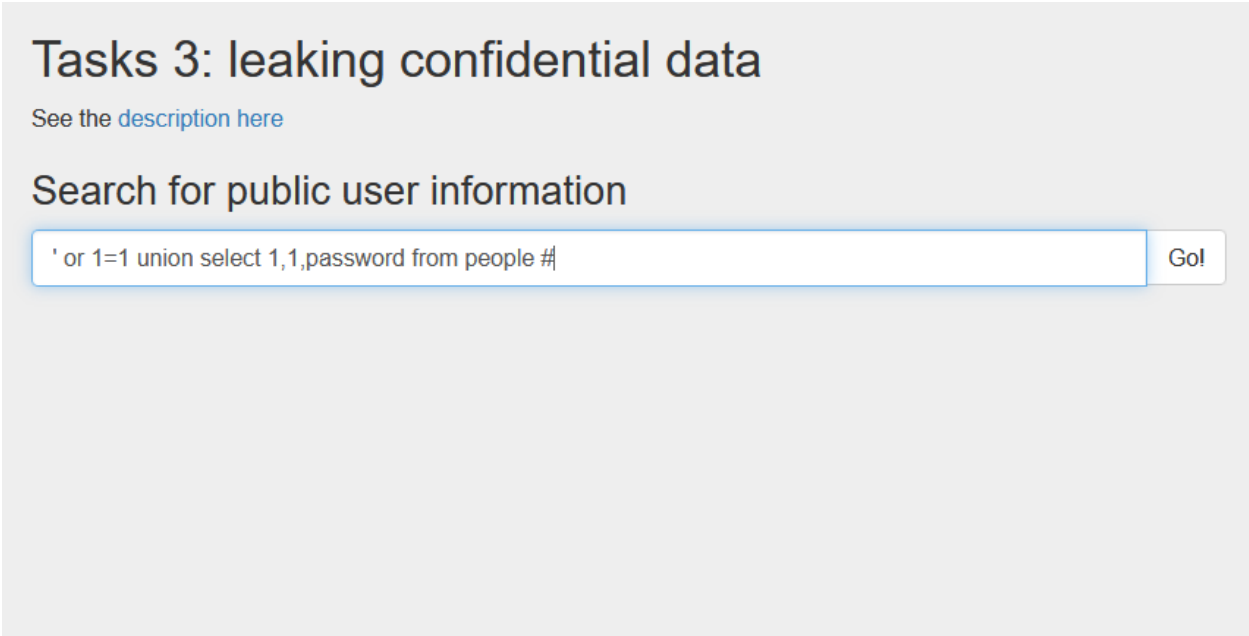


Task3: leaking confidential data

Problem: using an in-band SQL injection on a user search page to exfiltrate confidential data (passwords) from a table not used by the search query (people).

Method Used: I used a UNION-based SQLi. Firstly, with the same tautology I determined that it was possible to easily inject queries and I determined the number of columns in the original select, which I found out was 3 (name, last name and link). Secondly, I built the

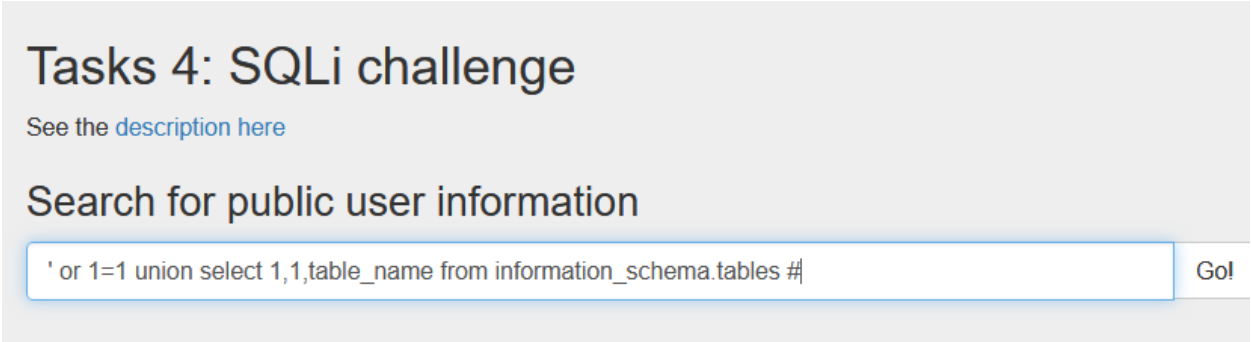
payload matching this column count. The payload is: `' or 1=1 union select 1,1,password from people #` . This input uses a tautology to make the first query always true and appended a new query which is SELECT 1, 1, password FROM people. The UNION operator combines the results of the original query with the results of the injected query. *1* was used as a placeholder to match the number of columns.

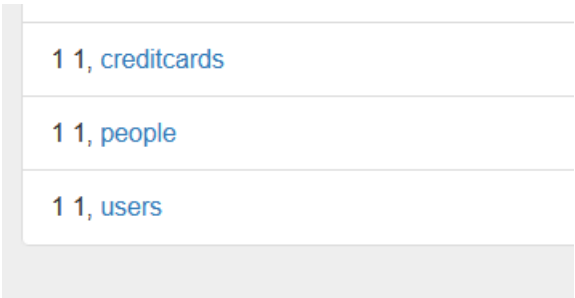


Task 4: final challenge

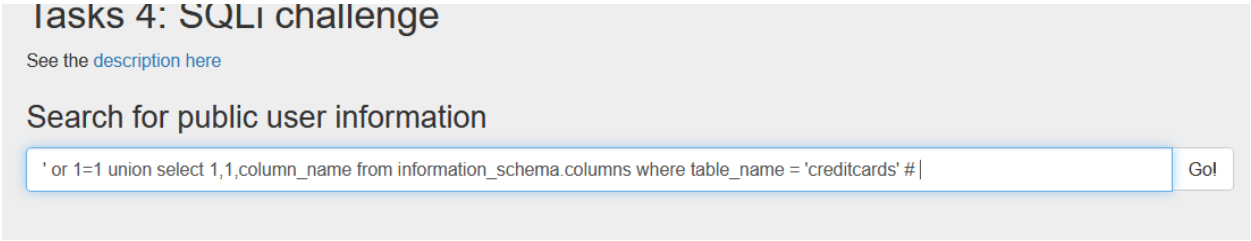
Problem: find a hidden flag located in an unknown table and column within the database.

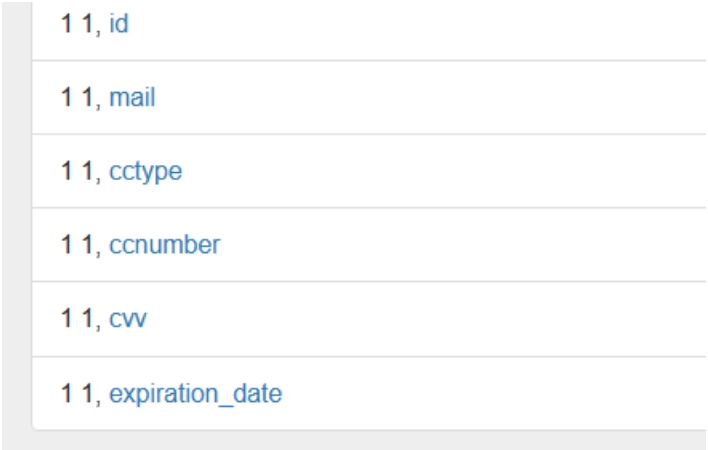
Method Used: This required a multi-step attack to first leak the database schema, using many union based attacks and the *information_schema*, that contains info about the DB structure (tables and columns). The first thing I did was dumping all the tables names of the schema. The payload is : `' or 1=1 union select 1,1,table_name from information_schema.tables #` which resulted in the leaking of the tables.





Then, since *people* was used in the original query, I thought that I had to search in *creditcards* or *users* (and I was right). So, bruteforcing, I tried with the first one. The goal of the previous query was to find the names of the table. Now I have to find the name of the columns. The payload is: `' or 1=1 union select 1,1,column_name from information_schema.columns where table_name = 'creditcards' #` . It resulted in the leakage *of id, mail, cctype, ccnumber, cvv, expiration_date*. Since *cvv* is a security information, I thought the information could be there, or at least in the *ccnumber*.





So, lastly, I queried cctype, ccnumber and cvv, obtaining the final flag. The payload is: `' or 1=1 union select cctype,ccnumber,cvv from creditcards #` . The result is the flag